

Kericho Primary Online Voting — Cookbook

Project: Kericho Primary Online Voting Management System

Author: Sharon Chepkirui · Supervisor: Grofry Rotich

Institution: Kenya Highlands University — School of Computing & Information Science (BSc IT)

Version: v1.1.0 · Date: 28 September 2026

Repo: <https://github.com/marvel-254/kericho-voting>

Live: <https://kericho-voting.onrender.com>

1. What this system does

A lightweight, browser-based election system for Kericho Primary School pupil-leadership elections. Pupils register for admin approval, log in with admission number + PIN, cast one ballot across five leadership positions, and cannot vote again. An authenticated admin sees live tallies, turnout, pending registrations, fraud reports and contact messages, and can reset the election.

Leadership positions (5)

- School Captain (Head Boy)
- School Captain (Head Girl)
- Assistant Head Boy
- Assistant Head Girl
- Games Captain

Each position has two candidates (seed data: 10 candidates total).

2. Tech stack & specs

Layer	Choice	Why
Server	Node.js **v24** built-in <code>http</code> + <code>node:sqlite</code> (<code>Database</code>)	Zero npm dependencies — <code>npm install</code> is trivial on Render
Client	Single <code>public/index.html</code> + <code>style.css</code> + <code>app.js</code> (vanilla)	No build step, fast to host
Storage	SQLite file <code> voting.db </code> (auto-created)	Survives restarts locally; ephemeral on Render free tier — auto-seed
Auth	Pupil: admission + SHA-256 PIN hash · Admin: username/p	No JWT library needed
Deploy	Render (Oregon, free, <code>npm install</code> / <code>node server.js</code>) + <code>npm</code>	Stable <code>https://kericho-voting.onrender.com</code>
CI	GitHub Actions — <code>setup-node@24</code> → <code>node --check</code> + <code>npm</code>	Gate for <code>autoDeploy: yes</code>

Project structure

```
voting/
server.js      # Node http server + SQLite + all APIs
package.json   # type:module, scripts: start / smoke
render.yaml    # Render blueprint (free, autoDeploy)
.gitignore     # voting.db, node_modules
COOKBOOK.md    # This file
COOKBOOK.pdf   # Printable version
public/
  index.html    # 13 views (landing, auth, ballot, admin...)
  style.css     # Design system: bento grids + skeu-brutalist landing + collapsible sidebar
  app.js        # Vanilla JS, Web Audio sounds, x-admin-token in sessionStorage
test/
  smoke.mjs     # 12 smoke tests (TC01–TC12) – local or BASE_URL live
.github/workflows/ci.yml
```

3. Prerequisites

- Node.js 24 (check: `node --version` → v24.x). Earlier Node lacks `node:sqlite`.
- Git (to clone), a terminal, and a browser.
- No npm install required locally — the server uses only built-ins. npm install on Render is a no-op but keeps the blueprint standard.

4. How to run the app locally

4.1 Clone & start (30 seconds)

```
git clone https://github.com/marvel-254/kericho-voting.git
cd kericho-voting
node server.js
# → Kericho Primary Voting System running on http://localhost:3000
# → Admin login: admin / admin123
```

Open `http://localhost:3000` in your browser.

The first run creates `voting.db` and seeds 10 voters (KP001-KP010, PIN 1234) + 10 candidates. Delete `voting.db` to re-seed.

4.2 Use a different port

```
PORT=4000 node server.js
# → http://localhost:4000
```

Render injects `PORT` automatically; the server binds `0.0.0.0:${PORT}` and reads `process.env.ADMIN_USERNAME / ADMIN_PASSWORD_HASH / DB_PATH / NODE_VERSION` if set.

4.3 Verify it works (local smoke)

```
# In a second terminal, with the server still running:
node test/smoke.mjs
# → Smoke tests against http://localhost:3000 – 12 cases – 12 passed

# Or against the live site:
BASE_URL=https://kericho-voting.onrender.com node test/smoke.mjs
```

4.4 Default accounts

Role	Admission / Username	PIN / Password	Notes
Pupil (seed)	`KP001` ... `KP010`	`1234`	Each can vote once; `HasVoted` ...
New pupil	Any new `KPxxx` you register	Your chosen PIN (≥ 4)	Needs admin approval before log...
Admin	`admin`	`admin123`	Logs in via *Admin* → token sto...

5. How the software works

5.1 Registration → approval flow

```
Pupil: POST /api/register {admissionNumber, name, class, pin}
  → row in voter_registrations as pending
Admin: GET /api/admin/registrations (pending first)
  → POST /api/admin/registrations/:id/approve (inserts into voters + approved)
  → POST /api/admin/registrations/:id/reject (rejected → pupil can re-apply)
Pupil: POST /api/login {admissionNumber, pin} → now succeeds → ballot
```

Validation: `adm` `^[A-Z0-9-]+\ $` 3–20, `name` ≥ 2 , `PIN` ≥ 4 , SHA-256 hex stored. Duplicate approved `adm` or pending application is rejected with a clear error.

5.2 Voting flow

```
GET / (landing) → Register or Login
Login: POST /api/login → returns {voter, candidates}
Ballot: 5 positions, exactly one radio per position
Submit: POST /api/vote {admissionNumber, selections: {Position: CandidateID x5}}
  → server checks: voter exists, not HasVoted, 5 selections, each CandidateID exists
  and Position matches, then BEGIN → +1 per candidate in votes → HasVoted=1 → COMMIT
  → frontend shows "Vote recorded" — re-vote blocked forever until admin Reset
```

5.3 Admin flows

- Results: GET /api/results (admin only) → {byPosition[{candidateId,name,class,votes,percent}], turnout, votedCount, totalVoters}. Percent per position = votes / total-votes-in-position. Dashboard renders bento KPIs + bars.
- Reset: POST /api/reset (admin only) → UPDATE votes SET 0 + HasVoted=0 in a transaction.
- Fraud & Contact: Public POST /api/fraud / POST /api/contact (validated) → admin-only GET /api/admin/fraud / GET /api/admin/contacts. Badges show pending counts.
- System Guide: Printable admin doc (Admin → System Guide + Print Guide → @media print / PDF via browser).

5.4 Security notes

- PINs never stored plaintext — SHA-256 hex. Admin password likewise (ADMIN_PASSWORD_HASH env override possible).
- Admin routes gated by x-admin-token random per login (issueAdminToken()). Missing/invalid → 401 Unauthorized — admin login required.
- Static assets served only under /public/ via join(cwd, pathname.slice(1)) (no path traversal).
- Cross-position vote injection rejected (Candidate Position !== selections Position).
- SQLite transactions BEGIN/COMMIT/ROLLBACK for vote + approval + reset.

6. API reference

All JSON. Base: <http://localhost:3000> (or live <https://kericho-voting.onrender.com>).

Method	Path	Auth	Body	Success
POST	/api/register	—	{admissionNumber, name, class, ...}	{ok:true, registration:{id,Adm...
POST	/api/login	—	{admissionNumber, pin}	{ok:true, voter, candidates[]}
POST	/api/vote	—	{admissionNumber, selections:{...}}	{ok:true}
POST	/api/fraud	—	{reporterName, admissionNumber}	{ok:true}
POST	/api/contact	—	{name, contact, message}	{ok:true}
POST	/api/admin/login	—	{username, password}	{ok:true, token}
GET	/api/results	x-admin-token	—	{ok:true, byPosition, turnout, ...}
GET	/api/admin/registrations	x-admin-token	—	{ok:true, registrations[]}
POST	/api/admin/registrations/:id/a...	x-admin-token	—	{ok:true}
POST	/api/admin/registrations/:id/r...	x-admin-token	—	{ok:true}
GET	/api/admin/fraud	x-admin-token	—	{ok:true, reports[]}
GET	/api/admin/contacts	x-admin-token	—	{ok:true, messages[]}
POST	/api/reset	x-admin-token	—	{ok:true}

Errors: {ok:false, error:"..."} . Admin without token → 401.

7. Database schema

```
voters(AdmissionNumber TEXT PK, PINHash TEXT, Name TEXT, Class TEXT, HasVoted INT)
```

```

candidates(CandidateID INT PK AUTOINCREMENT, Position TEXT, Name TEXT, Class TEXT)
votes(CandidateID INT PK FK, VoteCount INT)
voter_registrations(id INT PK, AdmissionNumber TEXT UNIQUE, Name TEXT, Class TEXT, PINHash TEXT, Status TEXT, CreatedAt TEXT, ReviewedAt TEXT)
fraud_reports(id INT PK, ReporterName TEXT, AdmissionNumber TEXT, Position TEXT, Details TEXT, CreatedAt TEXT)
contact_messages(id INT PK, Name TEXT, Contact TEXT, Message TEXT, CreatedAt TEXT)

```

Indexes: candidates(Position), voters(Class), voter_registrations(Status).

8. Frontend pages

- Landing — hero-brutal (skeu + brutalism, mustard borders, paper texture, ballot visual) — bento Why + 3-step + positions bento.
- Auth — bento: Login / Register each paired with an explainer card + voting imagery.
- After-registration — pending confirmation with Go to Login / Guide.
- Onboarding — 5-step bento: Register → Wait → Login → Pick one per position → Submit.
- Report Fraud / Contact Us — bento forms, admin-only review.
- Terms / Privacy — legal copy.
- Ballot — bento--ballot 2-col positions, exactly one per position.
- Admin — bento KPIs + bento results, tabs: Results / Registrations / Fraud / Messages / Guide. Settings tab: codebase downloads. Hamburger collapsible sidebar with voting ballot+check icon, overlay + Esc to close. Sounds (Web Audio tap/success/error/nav/submit, / toggle persisted).

Footer: Built by Sharon (demo hints and School of Computing footer removed).

9. Deploy & CI

- Repo: marvel-254/kericho-voting, branch main, autoDeploy: yes on Render.
- Render: service kericho-voting (Oregon, free, npm install / node server.js, NODE_VERSION=24). Blueprint render.yaml at repo root.
- CI: .github/workflows/ci.yml — Node 24, node --check server.js + node --check public/app.js + BASE_URL smoke (12 cases). Runs on push/pull_request to main.
- Codebase release: GitHub release codebase-v1.1.0 with assets kericho-voting-codebase.zip + COOKBOOK.pdf (also COOKBOOK.md). Admin Settings → Codebase links to <https://github.com/marvel-254/kericho-voting/releases/download/codebase-v1.1.0/....> Zip is credential-free (no voting.db, no rnd_gho_tokens, no .env, no node_modules).

10. Troubleshooting

Symptom	Fix
`node:sqlite` not found / `DatabaseSync` undefined	Use Node 24 (`node --version` must be `v24.x`).
`Invalid admission number or PIN`	Pupil not approved yet — check Admin → Registrations → Approve.
`You have already voted`	By design — one vote per pupil until admin Reset.
`401 Unauthorized — admin login required`	Log in via Admin; token lives in `sessionStorage` — re-login if expired.
Cold start on live site (blank 10s)	Free tier sleeps — wait a few seconds and refresh.
DB resets after deploy	Expected on free tier — `seedIfEmpty()` re-creates seed voters/candidates.

Built to be simple, fair and neat — keep the admin credentials safe.